

Day 2 – Core Models: Publisher, Game, GameKey

Goals: Define the core data model and get it into the database.

games/models.py:

```
from django.db import models
from django.contrib.auth.models import User

class Publisher(models.Model):
    name = models.CharField(max_length=200)
    webhook_url = models.URLField()
    webhook_secret = models.CharField(max_length=64)
    user = models.OneToOneField(User, on_delete=models.CASCADE)

    def __str__(self):
        return self.name

class Game(models.Model):
    title = models.CharField(max_length=200)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    price = models.DecimalField(max_digits=6, decimal_places=2)

    def __str__(self):
        return self.title

class GameKey(models.Model):
    STATUS_CHOICES = [('active', 'Active'), ('expired', 'Expired')]

    key_string = models.CharField(max_length=50, unique=True)
    game = models.ForeignKey(Game, on_delete=models.CASCADE)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES, de
    expires_at = models.DateTimeField()
    owner = models.ForeignKey(User, on_delete=models.CASCADE, null=True)

    def __str__(self):
        return self.key_string

python manage.py makemigrations
python manage.py migrate
```

Register all models in games/admin.py:

```
from django.contrib import admin
from .models import Publisher, Game, GameKey

admin.site.register(Publisher)
```

```
admin.site.register(Game)
admin.site.register(GameKey)
```

Instructor Teaching Plan

Time Breakdown (90 min)

Segment	Duration	Activity
ORM mental model	20 min	Explain: Django models = Python classes = database tables. Draw the entity diagram: User $\longleftrightarrow$ Publisher $\longleftrightarrow$ Game $\longleftrightarrow$ GameKey. Discuss the relationships before writing code.
Write models live	25 min	Type out Publisher, Game, GameKey with students. Pause on each field type and explain the choice.
Migrations deep dive	15 min	Run makemigrations, open the generated file, read it aloud. Run migrate. Check with <code>python manage.py dbshell</code> or DB browser. Students should understand migrations are <i>versioned schema changes</i> , not magic.
Django admin	15 min	Register models, create a superuser, explore the admin. Create a sample Publisher and Game through the UI.
Q&A + checkpoint	15 min	Verify all 3 models visible and editable in admin.

Key Concepts to Introduce

- **ORM = Object-Relational Mapper** — Python objects represent rows, class attributes represent columns.
- **ForeignKey vs OneToOneField** — FK: one publisher can have many games (one-to-many). O2O: one user can have at most one publisher profile.
- **on_delete=CASCADE** — deleting a Publisher deletes all their Games. Contrast with SET_NULL and PROTECT. Ask: "What should happen to GameKeys if we delete a Game?"
- **null=True vs blank=True** — null is database-level (allows NULL in column), blank is validation-level (allows empty string in forms/serializers). Common combo: null=True, blank=True for optional fields.
- **choices** — enforced at the Django layer, not at the database level. The DB stores the raw string 'active'/'expired'.
- **Migrations** — two steps: makemigrations (create the migration file) → migrate (apply it to the DB). The migration file should be committed to git.

Common Mistakes to Address

- **Missing __str__** — the admin shows "Publisher object (1)" everywhere without it. Add __str__ first, make it a habit.
- **Confusing null and blank** — students often add only blank=True for optional fields and then get database-level errors when saving.
- **Forgetting 'games' in INSTALLED_APPS** — makemigrations silently produces no output and students are confused. Teach: always check INSTALLED_APPS first.
- **Running migrate without makemigrations** — nothing happens. Emphasize the two-step sequence.
- **Editing migrations by hand** — discourage this. Always re-run makemigrations

after changing models.

? Check for Understanding

"Why does `GameKey.owner` use `null=True`, `blank=True` but `GameKey.game` does not?"

Answer: `GameKey.owner` is optional because a game key can exist (e.g., pre-loaded by a publisher) before any user purchases it. Thus, the database field must allow `NULL` (`null=True`) and API/form validation must allow it to be empty (`blank=True`). Conversely, a `GameKey` must always belong to a specific game, so `GameKey.game` is mandatory and cannot be null.

"What's the difference between `makemigrations` and `migrate`? What does each file represent?"

Answer: `makemigrations` scans model definitions for changes and creates new, numbered migration files (e.g., `0001_initial.py`), which are blueprints describing *how* to modify the database schema. `migrate` actually executes those blueprints against the database, applying the changes to create/modify tables. The migration files represent versioned history of the schema and must be committed to git.

"If we delete a `Publisher`, what happens to their `Game` records? What about their `GameKey` records?"

Answer: Because `Game.publisher` uses `on_delete=models.CASCADE`, deleting a `Publisher` automatically cascades and deletes all related `Game` records. In turn, because `GameKey.game` also uses `on_delete=models.CASCADE`, deleting those `Game` records will automatically cascade and delete all associated `GameKey` records.

"Why is `key_string` marked `unique=True`?"

Answer: A game key represents a single, unique digital asset. If it were not marked `unique=True`, the database could accidentally store duplicate key strings, leading to the same key being issued to multiple users, violating licensing rules and causing validation collisions.

✓ Day Checkpoint

`python manage.py migrate` completes with no errors. Student navigates to `/admin`, logs in, and can create a `Publisher`, a `Game` linked to it, and a `GameKey` linked to the game — all through the admin UI.